

AFRILANG Stdlib

std/c/graphfloyd

Bibliothèque AFRILANG `std/c/graphfloyd` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : floydWarshall,

```
`import "std/c/graphfloyd" · `use graphfloyd`
```

- `floydWarshall(adj list number, n number) ? list number`
- `pairDistance(adj list number, n number, a number, b number) ? number`
- `closureReachable(adj list number, n number) ? list number`
- `graphDensity(adj list number, n number) ? number`
- `avgShortestPath(adj list number, n number) ? number`
- `transitiveClosure(adj list number, n number) ? list number`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/graphfloyd` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : floydWarshall,

```
`import "std/c/graphfloyd"` · `use graphfloyd`  
- `floydWarshall(adj list number, n number) ? list number`  
- `pairDistance(adj list number, n number, a number, b number) ? number`  
- `closureReachable(adj list number, n number) ? list number`  
- `graphDensity(adj list number, n number) ? number`  
- `avgShortestPath(adj list number, n number) ? number`  
- `transitiveClosure(adj list number, n number) ? list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphfloyd"  
use graphfloyd
```

Niveau : complex · Catégorie : Modules complex (c/)

Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? floydWarshall(adj list number, n number) ? list  
? pairDistance(adj list number, n number, a number, b number) ? number  
? closureReachable(adj list number, n number) ? list  
? graphDensity(adj list number, n number) ? number  
? avgShortestPath(adj list number, n number) ? number  
? transitiveClosure(adj list number, n number) ? list
```

4. Exemples d'utilisation

```
import "std/c/graphfloyd"  
use graphfloyd  
  
create result = floydWarshall(/* args */)   
say result  
  
create result = pairDistance(/* args */)   
say result  
  
create result = closureReachable(/* args */)   
say result  
  
create result = graphDensity(/* args */)   
say result  
  
create result = avgShortestPath(/* args */)   
say result  
  
create result = transitiveClosure(/* args */)   
say result
```

5. Bonnes pratiques

? Préférez `import "std/c/graphfloyd"` en tête de fichier.
? Lancez `afirilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module graphfloyd  
  
function floydWarshall(adj list number, n number) returns list number  
end
```

```
function pairDistance(adj list number, n number, a number, b number) returns number
end

function closureReachable(adj list number, n number) returns list number
end

function graphDensity(adj list number, n number) returns number
end

function avgShortestPath(adj list number, n number) returns number
end

function transitiveClosure(adj list number, n number) returns list number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/graphfloyd` (tier complex, category complex). Exports 6 function(s): floydWarshall, pairDistance

```
`import "std/c/graphfloyd" . `use graphfloyd`  
- `floydWarshall(adj list number, n number) ? list number`  
- `pairDistance(adj list number, n number, a number, b number) ? number`  
- `closureReachable(adj list number, n number) ? list number`  
- `graphDensity(adj list number, n number) ? number`  
- `avgShortestPath(adj list number, n number) ? number`  
- `transitiveClosure(adj list number, n number) ? list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphfloyd"  
use graphfloyd
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? floydWarshall(adj list number, n number) ? list  
? pairDistance(adj list number, n number, a number, b number) ? number  
? closureReachable(adj list number, n number) ? list  
? graphDensity(adj list number, n number) ? number  
? avgShortestPath(adj list number, n number) ? number  
? transitiveClosure(adj list number, n number) ? list
```

4. Usage examples

```
import "std/c/graphfloyd"  
use graphfloyd  
  
create result = floydWarshall(/* args */)   
say result  
  
create result = pairDistance(/* args */)   
say result  
  
create result = closureReachable(/* args */)   
say result  
  
create result = graphDensity(/* args */)   
say result  
  
create result = avgShortestPath(/* args */)   
say result  
  
create result = transitiveClosure(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/graphfloyd"` at the top of your file.  
? Run `afirang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module graphfloyd  
  
function floydWarshall(adj list number, n number) returns list number  
end
```

```
function pairDistance(adj list number, n number, a number, b number) returns number
end

function closureReachable(adj list number, n number) returns list number
end

function graphDensity(adj list number, n number) returns number
end

function avgShortestPath(adj list number, n number) returns number
end

function transitiveClosure(adj list number, n number) returns list number
end
end
```